Haven: A Shared-Household App for Roommates

Team Members: Chris Mary Benson (U56085268), Nandana Shashi (U05556171)
Course: CS 501 E1 Mobile App Development
Repository Link: https://github.com/nanda002/Haven.git

1. Problem Statement

Roommates sharing a home often struggle to manage shared expenses and household chores fairly. Bills get split inconsistently, payments are forgotten, and chores fall unevenly on one or two people. Because these issues are usually tracked informally, through group chats, or scattered payment apps, there's no single, transparent record of who owes what or who has done what. This lack of visibility leads to resentment, awkward confrontations, and strained relationships between roommates.

We propose 'Haven', a shared household management app that serves as a centralized platform for logging and tracking both shared expenses and chores among all occupants living in the same household.

2. Target Users, Proposed Value, and Scope

Target users: Roommates sharing an apartment or house, who deal with shared financial and operational friction every week. We assume each user lives with at least one other person, that roommates share expenses on a regular basis, and that they divide household chores among themselves.

Proposed value: Roommates currently coordinate through disconnected tools such as group chats, Splitwise-style apps, whiteboards, and memory. Haven brings chores, groceries, balances, and rent reminders into one app, so everyone can see who owes what and who has done what.

Haven will aim to bring everyday roommate responsibilities into one place. Users will be able to create or join a household, manage shared expenses and rent, maintain a common grocery list and automate the assignment and rotation of daily and weekly chores.

2.1 Features of '*Haven*'

- **Expenses**: Roommates can log shared costs like rent, utilities, and groceries, and split them equally or by custom amounts. Haven will track who owes whom and keeps a clear payment history, so balances never get lost or forgotten.
- **Chores**: Household tasks can be assigned to specific roommates or set on a rotating schedule, with due dates and reminders (daily/weekly). Completed chores are marked off and visible to everyone, keeping the workload fair and transparent.
- **Common Grocery List**: A shared, real-time list where any roommate can add items the house needs. Whoever does the shopping can check items off, and the purchase can be logged directly as a shared expense.

- **Dashboard View of User's Pending Actions**: Each user gets a personalized home screen showing everything that relates to them, such as payments they owe, chores due soon, and grocery items to pick up.

2.2 Minimum Viable Product (MVP)

The points listed below represent the MVP of our application.

- **User login/signup**: Users can sign up, create a household, and invite roommates through a generated invite code. If users are already registered with a household, they are prompted directly to the household view with all the main features.
- **Expense Logging**: Users can add an expense with an amount, description, and who paid, and split it equally among roommates. Haven shows a simple running balance of who owes whom, and users can mark debts as settled.
- **Chore Distribution**: Users can create a chore and set it to repeat daily or weekly. Chores can be assigned to a roommate manually, or Haven can automatically rotate them among roommates for a specified number of weeks. Roommates can mark chores as complete, and the status is visible to the whole household.
- **Shared Grocery List**: A single list where roommates can add items and check them off once bought.
- **Dashboard View**: A home screen showing each user's outstanding payments and upcoming chores.
- **Basic CRUD operations** relating to the above features such as deleting an account, leaving or removing a user from the household, etc.

2.3 Planned Extensions (P1) and Stretch Goals (P2)

2.3.1 Planned extensions (P1):

- **QR Code Household Joining:** Roommates can join a household by scanning a QR code displayed on another member's phone, making setup faster than sharing codes.
- **Push Notifications:** Users receive reminders for upcoming chores, payments owed, and new expenses added to the household, when the application is closed/in the background.
- **Receipt Scanning:** Users can photograph a receipt, and Haven digitizes it into a list of individual items and prices. Users can then select which items belong to the household and exclude personal purchases, and Haven creates a shared expense from only the selected items.

2.3.2 Stretch goals (P2):

- **Calendar View of Chores:** A calendar layout showing each roommate's chores across the day, week, or month, making it easy to see the full schedule and plan around it.
- **Location-Based Grocery Reminders:** When a user enters a grocery store, Haven sends a notification listing items still pending on the shared grocery list, prompting them to buy the items.

2.4 Non-Goals

- ➢ No in-app money transfers or bank and payment-app integration. Users settle up outside Haven and mark the debt as settled.
- ➢ No iOS or web version. Haven is an Android app only.
- ➢ No membership in more than one household per account.
- ➢ No in-app chat or messaging between roommates.
- ➢ No continuous location tracking. Location is used only for the optional grocery-store reminder.

3. User stories

Below are three user stories listed with the acceptance criteria for each, which details what the user would be able to do with that feature.

(i) **User story 1 (P0):** As a roommate, I want chores to repeat daily or weekly and rotate automatically so that the workload is shared fairly without anyone having to reassign them.

Acceptance Criteria:

- The user can create a chore and set it to repeat daily or weekly.
- The user can assign the chore manually or enable automatic rotation for a specified number of weeks.
- When rotation is enabled, the chore cycles through all household members in order.
- Any roommate can mark their chore as complete, and the status is visible to the whole household.

(ii) **User story 2 (P0):** As a roommate, I want to add items to a shared grocery list so that whoever goes shopping knows what the household needs.

Acceptance Criteria:

- Any household member can add an item to the list.
- Items added appear for all household members.
- The user can check off an item once it's purchased.
- Checked-off items are removed from the pending list.

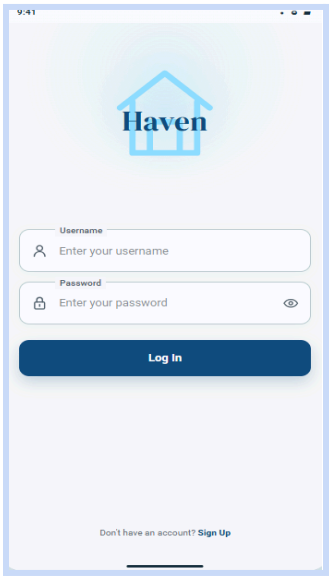
(iii) **User story 3 (P0):** As a roommate, I want a dashboard showing my pending actions so that I can quickly see what I owe and what I need to do.

Acceptance Criteria:

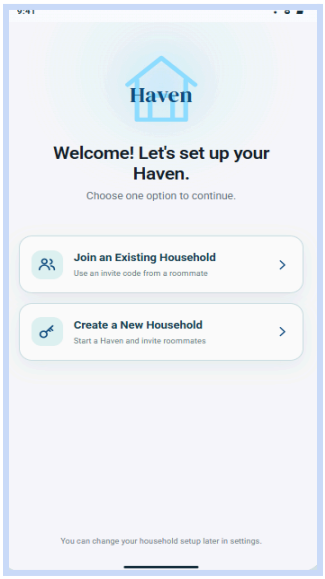
- The dashboard appears as the home screen after the user logs in.
- It shows all outstanding balances the user owes to other roommates.
- It shows the user's chores due today and in the upcoming week.
- Items are removed from the dashboard once a payment is settled or a chore is marked complete.

4. Wireframes

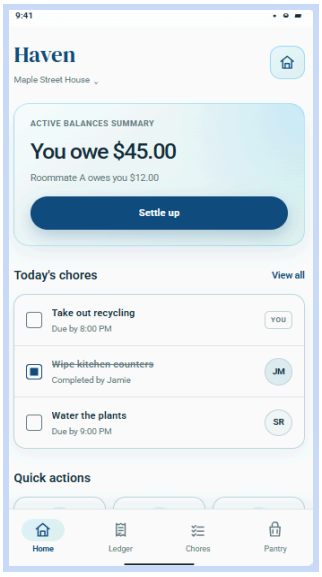
We present below some early wireframes of our application, designed using Figma. The UI is subject to change in further development of the application.



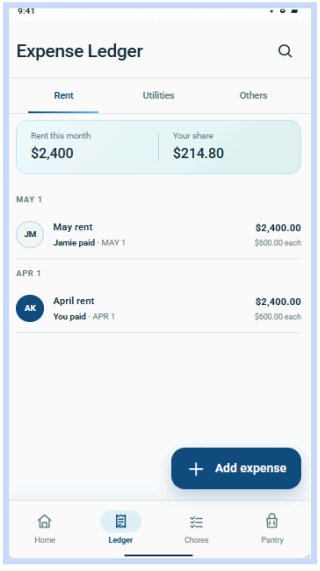
User login/signup screen



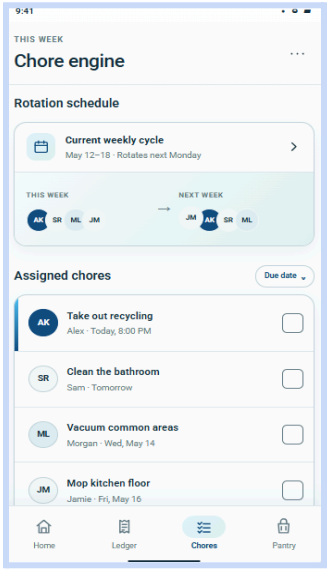
Prompt to join or create a household



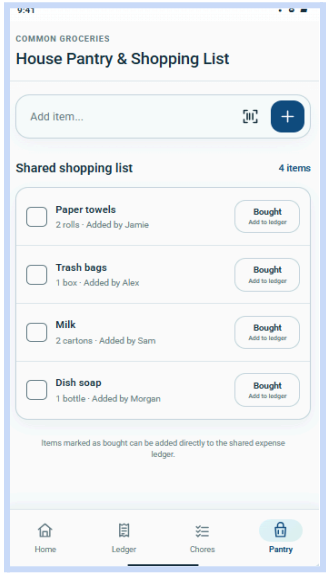
Dashboard view



Expense Logger and Tracker

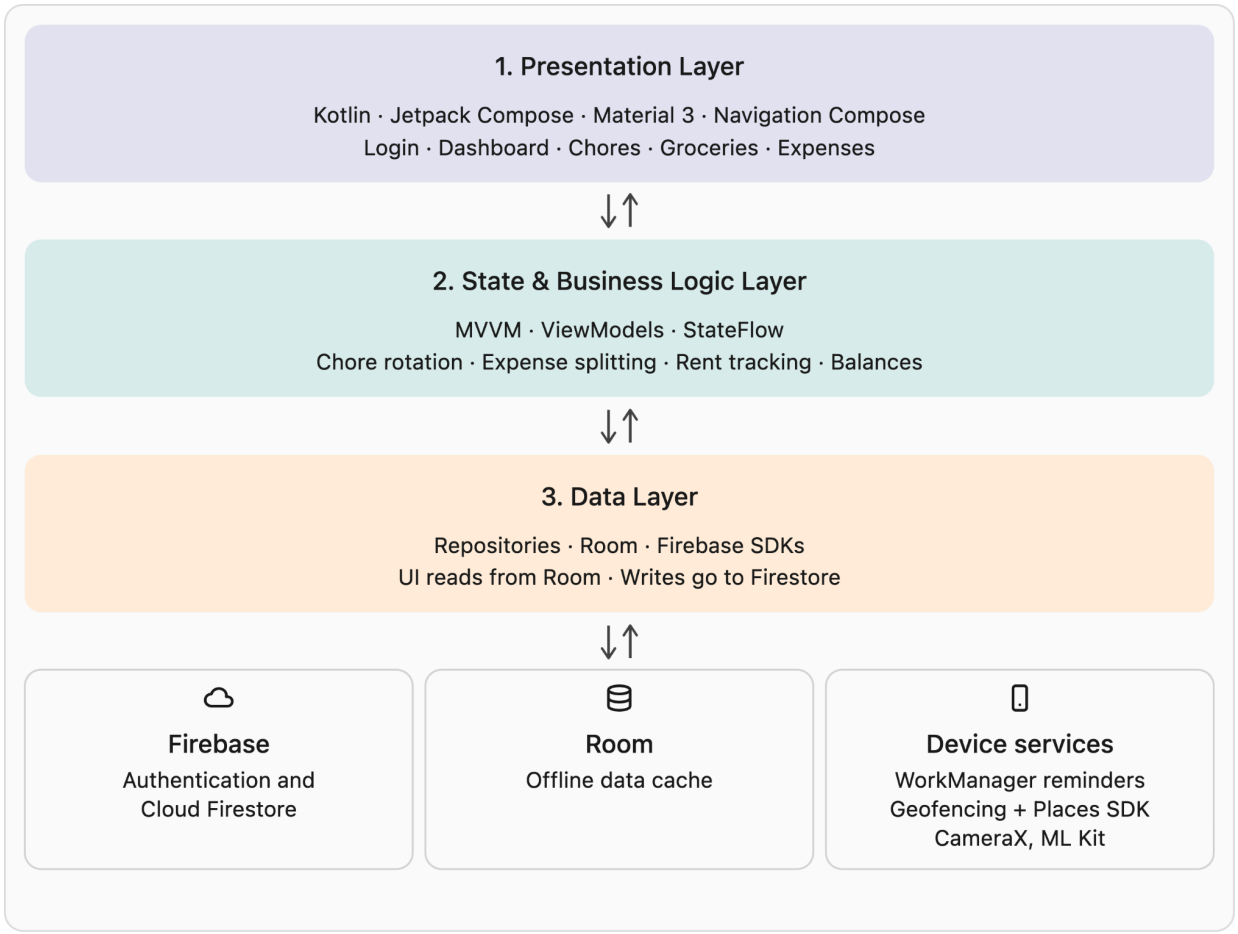


Chore Tab



Shared Grocery List

5. Planned Architecture/Tech Stack



5.1 Overview:

Haven will be an Android application built with Kotlin and organized in three layers (presentation, state and business logic, and data), backed by Firebase and on-device services. It will follow the MVVM pattern with a repository layer. Each layer will depend only on the one below it.

5.2 Layers:

- **Presentation Layer:** The screens users interact with (login, dashboard, chores, groceries, and expenses) will be built with Jetpack Compose and Material 3. Navigation Compose will handle movement between the four-tab bottom bar (Home, Ledger, Chores, Pantry)

- **State and Business Logic:** ViewModels will expose each screen's state through StateFlow, collected in a lifecycle-aware way, and will handle user actions. This layer will hold the core

logic: chore assignment and weekly rotation, even and uneven expense splitting, and the calculation of balances and rent.

- **Data Layer:** Repositories will give the ViewModels a single interface for household data. The UI will read from Room, and repositories will write to Cloud Firestore and keep Room updated from it. The Firebase SDKs will connect to the backend.

5.3 Backend and Device Services:

- **Firebase Authentication:** will handle sign-up and login with email and password.
- **Cloud Firestore:** will be the shared source of truth for households, members, chores, groceries, expenses, and rent records.
- **Room:** will mirror household data on the device, so screens load quickly, can be read offline, and can run the queries used for balance calculations.
- **WorkManager and NotificationCompat:** will schedule local reminder notifications for chores and rent payments.
- **Google Play services Location (Geofencing) and Places SDK for Android (context-aware feature):** will let users pick their usual grocery stores and will notify them with the pending grocery list when they arrive at one.
- **CameraX and Google ML Kit Text Recognition (stretch):** will capture receipts and extract the likely total on the device for the user to confirm.

5.4 Key Design Decisions

Data ownership and offline behavior: Cloud Firestore will be the shared source of truth. Writes will go directly to Firestore, which queues them while offline and applies them on reconnection. Snapshot listeners will mirror the data into Room, and the UI will read only from Room. Creating or joining a household and advancing the chore rotation require a connection, and the app will show a clear message when the device is offline.

Avoiding Conflicts: Several roommates may edit data at the same time, including offline, so conflicts will be handled by data type:

- Groceries and chores: each item will be its own Firestore document, so two roommates adding items will never touch the same record. Edits will be written to Firestore, which queues them while offline and applies them on reconnection. If two people change the same field, the last write to reach Firestore will be kept. Deleted items will be marked as deleted instead of removed, so every device and the history stay consistent. Advancing the chore rotation will happen only while online, because it uses a Firestore transaction.
- Expenses and balances: expenses and payments will be saved as immutable entries, and balances will be calculated from them rather than stored as one editable number. Amounts will be stored as integer cents to avoid rounding errors. Editing or deleting an expense will create a voiding or correcting entry, so history is preserved and balances are recalculated.
- Claims and rotation: actions that happen exactly once, such as joining with an invite code or advancing the chore rotation, will use Firestore transactions.

- Recurring rent: rent entries will use a deterministic document ID (for example, rent-2026-11), so simultaneous creation by several phones produces one record.

Members leaving or deleting accounts: Each expense will store a snapshot of its participants and shares. When a member leaves a household or deletes their account, their member record will be marked as "former" and shown as "Former member". Deleting an account will require re-authentication. Past expenses and balances will therefore stay intact.

New members and splits: A new member will be included only in expenses dated after they join. Each expense will store its own participant list, and even and uneven splits will be calculated from that list, with any leftover cents assigned deterministically.

Security: Firestore security rules will allow only household members to read or write that household's data. New members will join through a time-limited invite code: codes will be long and random, stored in Firestore with an expiry time, and Firestore rules will allow a user to join only by presenting a valid, unexpired code, checked against the server's time. Location will be requested only when the user enables the grocery reminder. The device's location will be checked on the device by Google Play services and not uploaded, and only the store locations the user selects will be stored. The app will request location (foreground and background, which Android requires for geofencing), notification (Android 13+), and camera permissions only when the related feature is enabled. Geofences will be re-registered when the app starts and after a device restart. The Places API key will be restricted to the app.

6. Risks and Mitigations

Type	Risk	Mitigation
Technical	Data goes out of sync across roommates, or offline edits overwrite each other	Per-item Firestore documents, immutable expense entries with derived balances, transactions for one-time actions
Technical	Geofencing and Places setup is complex (permissions, API key, device restarts)	Build it early on a real device, restrict the API key, re-register geofences, fall back to a "saved favorite stores" reminder if blocked
Technical	Split calculations are wrong, especially uneven splits	Integer cents, deterministic handling of leftover cents, unit tests
Usability	Roommates stop logging expenses because it takes too long	Quick add-expense form, "Bought" button that creates an expense from a grocery item, usability test with 3 to 5 roommates
Usability	Joining a household with a code is confusing or fails	Clear error messages for invalid or expired codes

Privacy	Financial and household data is exposed to non-members	Members-only Firestore rules, tested with the Firebase Emulator
Privacy	Location data is over-collected	Location requested only when the reminder is enabled, geofencing on the device, no continuous tracking
Schedule	Two people and about seven weeks cannot finish everything	Priorities (P0, P1, P2), an MVP feature freeze on November 22, and a cut rule that drops P2 items first

7. Milestone plan and Initial Backlog

7.1 Initial Backlog

Item	Priority	Owner
Repository and Firebase setup	P0	Both
Data model, Firestore and Room repositories	P0	Both
4-tab navigation and Compose theme	P0	Chris
Sign up, login, create/join household with invite code	P0	Nandana
Firestore security rules	P0	Chris
Chores: daily/weekly repeat, manual assignment, automatic rotation	P0	Chris
Chores screen (list, add, mark complete)	P0	Nandana
Expense logging with equal and custom splits	P0	Chris
Balances and settle-up	P0	Chris
Ledger screen	P0	Nandana
Shared grocery list	P0	Nandana
Dashboard view	P0	Nandana

Leave household and delete account	P0	Both
Testing, UI polish, bug fixes	P0	Both
Demo prep and project report	P0	Both
QR code household joining	P1	Nandana
Push notifications	P1	Chris
Receipt scanning	P1	Both
Calendar view of chores	P2	Both
Location-based grocery reminders	P2	Nandana

7.2 Milestone Plan

Weeks	Tasks
Oct 14 to Oct 20	Proposal Documentation, Repository Setup, Planning Tech Stack
Oct 21 to Oct 27	Data Model, Firebase & Room Setup, User Login/Signup & Joining household
Oct 28 to Nov 3	4 tab navigation, Expenses and Chore tab userflow implementation
Nov 4 to Nov 10	Dashboard view, Expenses and Chore tab userflow implementation, prepare for Update 1
Nov 11 to Nov 17	Common Grocery List, Push Notifications, QR code scanning
Nov 18 to Nov 24	Push Notifications, Receipt Scanning, Prepare for Update 2
Nov 25 to Dec 1	UI/UX cleanup, Testing, Bug fixes, Finishing any other pending work
Dec 2 to Dec 8	Project Report, Testing, Demo Prep

8. Team Collaboration Plan & Responsibilities

As a collaborative team of two, we will share all design and architecture decisions. To make sure we make steady progress, we have assigned each member primary focus areas, but we will both contribute to the entire codebase through code reviews and pair programming on the harder features, such as expense splitting and sync.

8.1 Roles

- **Nandana Shashi:** Repository and Firebase setup, user sign-up and login, household creation and joining, user facing screens, grocery-store reminder, rent tracking, UI polish.
- **Chris Mary Benson:** Repository and Firebase setup, Navigation, data model, Firestore and Room repositories, chore rotation logic, expense splitting, balances, push notifications, security rules.
- **Shared:** Compose theme, code review for every pull request, sprint planning, usability testing, demo preparation, and project report.

8.2 Communication Plan

- **Meetings:** We will hold two meetings each week to review progress and pair-program the features that need both of us.
- **Task Tracking:** GitHub Issues will hold the backlog, with one issue for each backlog item.
- **Scrum Master:** We will follow an agile approach with two-week sprints. The scrum master role will rotate between us every sprint.

8.3 Working Agreement

- Each task is done on its own branch and merged to main
- If a task will not be finished, we say so at least two days before the sprint ends so it can be reassigned or cut.
- Main must always build.

9. AI Usage and References

Generative AI (Claude) was used to brainstorm app name ideas, identify possible APIs and cloud services, identify potential risks, and help draft and refine this proposal. Generative AI (Figma) was used to create a rough draft of our wireframes.

- Reference 1: https://developers.google.com/ml-kit/guides
- Reference 2: https://developer.android.com/develop/ui/compose/notifications/create-notification
- Reference 3: https://developers.google.com/maps/documentation/android-sdk/location
- Reference 4: https://developers.google.com/maps/documentation/places/android-sdk/overview